



LA DESPENSA

MOLINA DE SEGURA

DOCUMENTACION PRACTICA RESTAURANTE
PHP

ADRIÁN VICENTE LÓPEZ

DESARROLLO WEB ENTORNO - SERVIDOR

2ºDAW

-CONTENIDO-

1.-INTRODUCCION.....	2
2.-DEFINICIONES DE ROLES Y PERFILES	2
3.-SPRINTS.....	2
4.-PROBLEMAS SURGIDOS.....	34
4.1- PROBLEMA CON CARRITO.....	34
4.2-PROBLEMA DE CERRAR SESION Y MESAS BLOQUEADAS	35
4.3-PROBLEMA AÑADIR AL CARRO UNA VEZ CERRADO EL PEDIDO.....	36
5.-BIBLIOGRAFIA.....	37

1.-INTRODUCCION

El presente trabajo consiste en el desarrollo de una aplicación web integral para la **gestión de un restaurante**, cuyo modelo de negocio y operativa son similares a los de grandes cadenas de restauración rápida como **McDonald's**. El proyecto se enfoca en digitalizar todo el **flujo de trabajo** de un establecimiento hostelero, desde que el **cliente se sienta a la mesa** hasta que se emite la **factura final**.

La práctica se ha organizado bajo el marco de trabajo **Scrum**, dividiendo el desarrollo en **cinco Sprints** o ciclos de entrega. Esta metodología permite construir el sistema de **forma incremental**, asegurando que cada fase aporte una funcionalidad **operativa y testeada**.

2.-DEFINICIONES DE ROLES Y PERFILES

El sistema está diseñado para interactuar con tres perfiles de usuario claramente diferenciados

- **Cientes:** Disponen de una interfaz para **registrarse**, seleccionar mesa, consultar la carta mediante un buscador y realizar pedidos en **tiempo real**, con la posibilidad de **añadir notas** personalizadas a sus productos.
- **Camareros:** Actúan como intermediarios para gestionar el **estado de las mesas**, convertir pedidos en **productos "servidos"**, generar cuentas automáticas en **PDF e imprimir los tickets físicos**.
- **Encargados:** Poseen el control administrativo del restaurante, gestionando el **stock** de productos, el **personal** (camareros), la supervisión de cuentas activas y la generación de **informes de ingresos**.

3.-SPRINTS

En este punto **indexaremos** los sprint para un manejo de la información **más fácil e incremental** de manera que en este mismo documento iremos añadiendo uno a uno para dejar toda la información en un **solo documento**

-CONTENIDO-

3.1-SPRINT 1	3
3.1.1 ESTRUCTURA DEL SITIO WEB.....	3
3.1.2 BASE DE DATOS.....	4
3.1.3 AUTENTIFICACION Y REGISTRO	4
3.1.4 DIFERENCIACION DE ROLES	5
3.1.5 SEGURIDAD Y LOGOUT	5
3.1.6 CREACIÓN Y BLOQUEO DE CATEGORIAS, PRODUCTO Y STOCK.....	6
3.2-SPRINT 2	9
3.2.1 INTERFAZ DE CLIENTE.....	9
3.2.2 CONTROL DE STOCK.....	12
3.2.3 IMPLEMENTACION DE NOTAS DE PRODUCTO.....	14
3.2.4 BUSCADOR DE PRODUCTOS	17
3.2.5 MARCADO DE PRODUCTOS VISTA CAMARERO	18
3.3-SPRINT 3	20
3.3.1 GENERACION DE CUENTAS Y CIERRE DE MESAS	20
3.3.2 FACTURAS DE CLIENTE.....	23
3.4-SPRINT 4	27
3.4.1 SERVICIOS DE ENVIO A COCINA Y TICKETS.....	27
3.4.2 CONTROL DE USUARIOS REGISTRO Y BLOQUEO	29
3.4.2 GENERACION DE INFORMES ESTADISTICOS.....	31

3.1-SPRINT 1

3.1.1 ESTRUCTURA DEL SITIO WEB

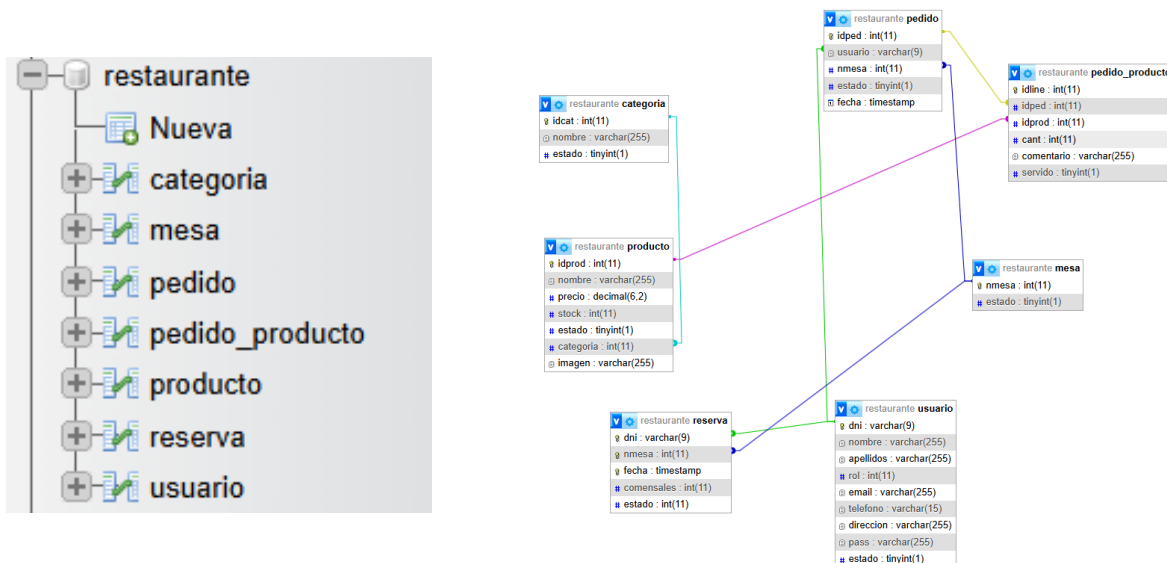
En la estructura de mi proyecto consta de por el momento varias páginas, por ahora vamos a resaltar primero que el diseño de la web ha sido creado en **base a una plantilla** que por internet y en ese concepto que yo quería como **minimalista e intuitivo** sin grandes cambios de colores ni formas diferentes y que estuviera todo más **recogido en un mismo sitio**, he ido creando pagina a pagina todos los elementos verificando que tuvieran características responsive con Bootstrap.

Por el momento en la estructura de paginas que llevo, se diferencia de **cuatro partes**, la primera y seria la raíz, a continuación, hare un pequeño desglose y una captura para que se vea y las otras **tres** las de **encargado, cliente y camarero**.

En **la raíz** tenemos las carpetas de encargado, cliente, camarero y la carpeta de imágenes, además de las páginas de :

```
restauVM
├── ┌─ encargado/
│   ├── añadircategorias.php
│   ├── añadirproductos.php
│   ├── añadirusuario.php
│   ├── bloquearcategoria.php
│   ├── bloquearproducto.php
│   ├── bloquearusuario.php
│   ├── categorías.php
│   ├── productos.php
│   ├── editarproductos.php
│   ├── indexEncargados.php
│   ├── modperfil.php
│   └── nav.php
├── ┌─ cliente/
├── ┌─ camarero/
├── ┌─ img/
│   ├── index.php
│   ├── login.php
│   ├── logout.php
│   ├── registro.php
│   ├── footer.php
│   ├── head.php
│   ├── nav.php
│   ├── conexión.php
│   ├── carta.php
│   ├── quienessomos.php
│   ├── ubicación.php
│   ├── términos.php
│   └── styles.css
```

3.1.2 BASE DE DATOS



3.1.3 AUTENTIFICACION Y REGISTRO

- login.php

esta es la página de **lógica del registro**, o sea la que hará que cuando ingreses en la página haga una consulta a la base de datos con el **DNI** y la **contraseña**, busque que usuario eres y si estas en la base de datos, te redirijan a una página u otra según tu rol, si estuvieras ya con la **sesión abierta** haría lo mismo al intentar **ingresar directamente** ahí.

- registro.php

esta página se encarga de hacer una **inserción** en la base de datos con todos los datos que ha introducido el **usuario**, y que de esa forma quede **registrado** en la base de datos. **Pd:** desde el registro **solo** se pueden **crear clientes** o sea que todo lo que entre desde esta página tendrá un **rol predeterminado de 0** (que es como se indica al cliente). Tanto la lógica como la parte visual, **están integrados en la misma página** de manera que el formulario se envía a la misma página y recoge todos los datos por el método **"POST"** y de esta forma pueda dejarlos como **'value'** en cada **"input"** y así si el usuario se equivoca en la inserción o pone alguna contraseña mal pues **no tiene que volver** a repetir todos los campos, sino tan solo los que le faltaron o donde tuvo el error.

3.1.4 DIFERENCIACION DE ROLES

esta página es la que tiene la lógica de redirigir el flujo según el rol, o sea recoge los datos mediante el “POST” del “login” para después hacer una **consulta y verificar que usuario** es el que tiene ese **DNI** y esa **contraseña, obtiene el rol** y según su rol lo **redirige** a una u otra página. También utilizo la misma lógica para redirigir si ya esta seteada esta variable de sesión de su rol y redirigirlo, para evitar que con un inicio de sesión activo pueda navegar por una página donde no debería, o sea que por rol estuviera restringido.

los roles van desde el cero hasta el dos, siendo el **cero el cliente**, el **uno, el camarero** y por último el **dos sería el encargado**

```
// Redirigir según rol
switch ($row['rol']) {
    case '0':
    case 0:

        header('Location:clientes/indexClientes.php');
        exit();
    case '1':
    case 1:

        header('Location:camareros/indexCamareros.php');
        exit();
    case '2':
    case 2:

        header('Location:encargados/indexEncargados.php');
        exit();
    default:
        // Rol desconocido, cerrar sesión por seguridad mínima
        session_unset();
        session_destroy();
        echo "Rol no reconocido.";
        exit();
}
```

```
<?php
session_start();

include("conexion.php");

// Si ya hay sesión, redirigir según rol
if (isset($_SESSION['rol'])) {
    if ($_SESSION['rol'] == 0) {
        header('Location:clientes/indexClientes.php');
        exit();
    }
    if ($_SESSION['rol'] == 1) {
        header('Location:camareros/indexCamareros.php');
        exit();
    }
    if ($_SESSION['rol'] == 2) {
        header('Location:encargados/indexEncargados.php');
        exit();
    }
}

// Procesamiento del formulario
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $dni = strtoupper(trim($_POST['dni']));
    $pass = trim($_POST['pass']);

    // Consulta simple: buscar usuario con dni y pass (texto plano)
    $consulta = "SELECT * FROM usuario WHERE dni = '$dni' AND pass = '$pass' LIMIT 1";
    $result = mysqli_query($conn, $consulta);

    if ($result && mysqli_num_rows($result) == 1) {
        $row = mysqli_fetch_array($result);

        //Comprobamos que el usuario este bloqueado
        if ($row['estado'] == 1) {
            $_SESSION['error_login'] = "Tu cuenta está bloqueada. Contacta con un encargado.";
            header('Location:index.php');
            exit();
        }

        // Crear sesión
        $_SESSION['dni'] = $row['dni'];
        $_SESSION['nombre'] = $row['nombre'];
        $_SESSION['rol'] = $row['rol'];
        $_SESSION['email'] = $row['email'];
        $_SESSION['telefono'] = $row['telefono'];
        $_SESSION['direccion'] = $row['direccion'];
        $_SESSION['apellidos'] = $row['apellidos'];
        $_SESSION['estado'] = $row['estado'];
        $_SESSION['pass'] = $row['pass'];
    }
}
```

3.1.5 SEGURIDAD Y LOGOUT

Esta página se encarga de la **salida de la sesión (logout)**, que sea segura y que **borre** todas las variables de sesión. Por último, redirige al usuario al **inicio**.

```
<?php
session_start();
include("conexion.php");

session_unset();
session_destroy();

// Redirigir al usuario a la página de inicio
header('Location:index.php');
exit();

?>
```

Y en la siguiente diapositiva se muestra el archivo de **seguridad**, el cual hice una **pequeña función** ("proteger()") en la cual llamo después del "include" del archivo de seguridad en cada página, y de esta forma me permite tener **un solo archivo** de seguridad **en la raíz** y no tener uno en cada apartado diferenciando en cada uno el rol, de esta forma tan **solo metiéndole el parámetro** que sea si es **0,1 o 2** y si es diferente al parámetro pues redirige al index. Al lado de la diapositiva de **seguridad** pondré otra con un ejemplo de un **include y su función**.

```
<?php
session_start();

function proteger($rolpermitido){

    // Verificar si está logeado Y si el rol es uno de los roles permitidos que seran los del parametro

    if (!isset($_SESSION['rol']) || $_SESSION['rol'] != $rolpermitido) {
        header('Location:../index.php'); // Redirigir si no cumple el requisito
        exit;
    }
}

?>
```

```
<?php
include("../seguridad.php");
proteger(2);
include("../conexion.php");
?>
```

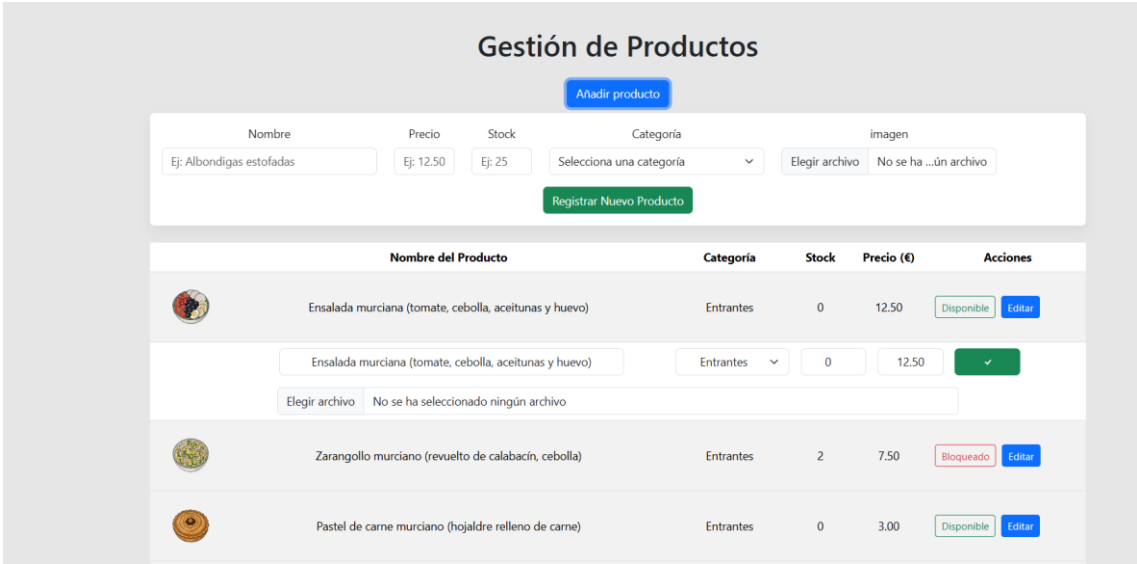
Como podemos observar siempre hacemos **referencia a la raíz** y poniendo **el número que sería el rol permitido**, simplemente redirige a una página u otra.

3.1.6 CREACIÓN Y BLOQUEO DE CATEGORIAS, PRODUCTO Y STOCK

En este apartado detallaremos el proceso **de creación, edición y bloqueo** de categorías productos y stock.

En la página de encargado tenemos varios apartados como son:

- **Productos:** página **de listado de productos**, desde aquí se pueden **añadir, editar o bloquear** productos



Gestión de Productos

Añadir producto

Nombre: Ej: Albondigas estofadas | Precio: Ej: 12.50 | Stock: Ej: 25 | Categoría: Selecciona una categoría | Imagen: Elegir archivo | No se ha ...ún archivo

Registrar Nuevo Producto

Nombre del Producto	Categoría	Stock	Precio (€)	Acciones
 Ensalada murciana (tomate, cebolla, aceitunas y huevo)	Entrantes	0	12.50	Disponibile Editar
Ensalada murciana (tomate, cebolla, aceitunas y huevo)	Entrantes	0	12.50	✓
 Zarangollo murciano (revuelto de calabacín, cebolla)	Entrantes	2	7.50	Bloqueado Editar
 Pastel de carne murciano (hojaldre relleno de carne)	Entrantes	0	3.00	Disponibile Editar

De nuevo como vemos en la diapositiva, **el collapse muy presente** para resolver el que **no te mande a otra pagina para editar el producto ni para añadir un producto** sino que con un collapse y **precargando todos los datos en los formularios** he decidido resolver todo eso de esta forma que de esta forma se queda con un diseño **mas compacto**, muchas menos paginas **y mas minimalista** a la par que da una sensación de mas profesionalidad y tecnicismo aunque en realidad es algo más sencillo que hacer que redirijan a otras paginas y hacer otros **diseños exclusivamente** para el mismo propósito. También podemos observar que el producto bloqueado desde ahí ya **no sale en la carta del cliente**.

En esta diapositiva se ve el ejemplo, que al **estar bloqueado** no se muestra el producto. Esto lo resolví poniéndole que solo haga el bucle al mostrar de los productos que como **estado** tengan el **0 (disponible)**.



- **Categorías:** Esta página lista todas las categorías y también se podrá bloquear o añadir nuevas categorías de manera que cuando bloquee una categoría, también quedaran bloqueados todos sus productos y por tanto no saldrá ni la categoría ni los productos en la carta.

Gestión de las categorías

[Añadir categoría](#)

Nombre

Ej: Cafes e infusiones

[Registrar Nueva categoría](#)

Nombre Categoría	Acciones
Entrantes	Disponible
Platos Principales	Disponible
Postres	Disponible
Bebidas	Bloqueado

Este tipo de **bloqueo** lo marcamos de forma diferente para que se quede diferenciado que es en **bloque** y **no individual** como el anterior ejemplo.

	Vino tinto de Jumilla (copa)	Bebidas	90	3.00	Cat.Bloqueada	Editar
	Cerveza artesanal murciana	Bebidas	90	3.50	Cat.Bloqueada	Editar
	Agua mineral	Bebidas	90	1.50	Cat.Bloqueada	Editar
	Sangria casera	Bebidas	90	4.50	Cat.Bloqueada	Editar
	Refrescos	Bebidas	90	2.10	Cat.Bloqueada	Editar

Postres

	Paparajotes murcianos (hojas de limón rebozadas en masa tipo churro)	4.50 €
	Arroz con leche de la abuela	4.00 €
	Tocino de cielo	4.20 €
	Tarta de limón murciano	5.00 €
	Pan de Calatrava	4.00 €

Bebidas

No hay productos disponibles en esta categoría.

y cómo podemos observar **tampoco** en la carta nos sale como disponible, poniéndonos que **no hay productos** de esa categoría.

Cabe resaltar que tanto los productos como los usuarios y las categorías, se puede ver que **no está puesta** la opción de **bloquear de inicio** o no, por un principio básico de que **no vas a crear algo solo para bloquearlo** y por tanto se supone que **de inicio debería ser visible**, si mas adelante se decide que no pues para eso se crean los **botones de bloqueo**.

3.2-SPRINT 2

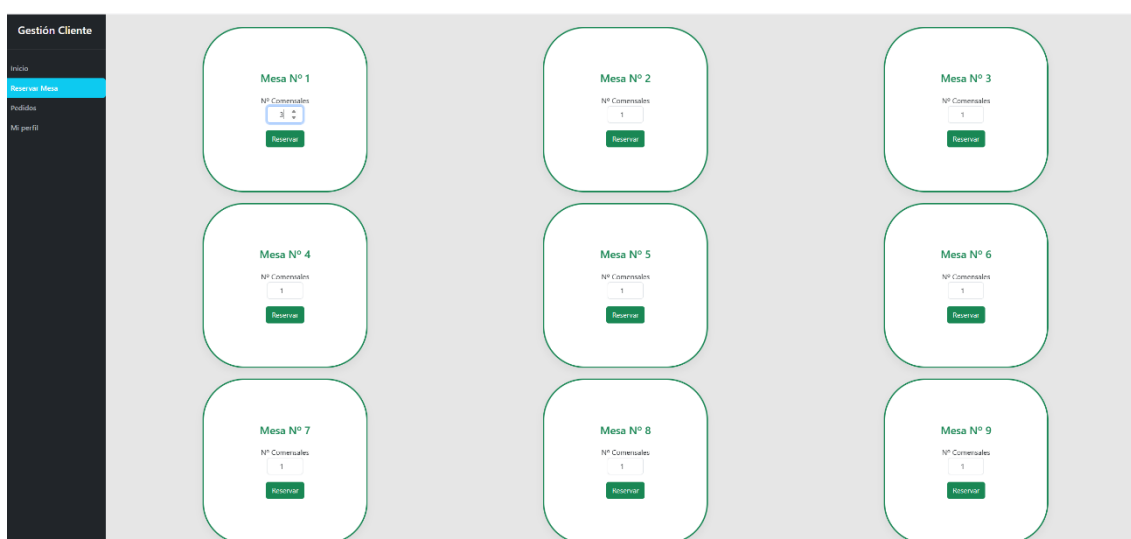
3.2.1 INTERFAZ DE CLIENTE

Para este apartado en el que es el cliente, decidí dejar **cuatro pestañas** con navs, que son:

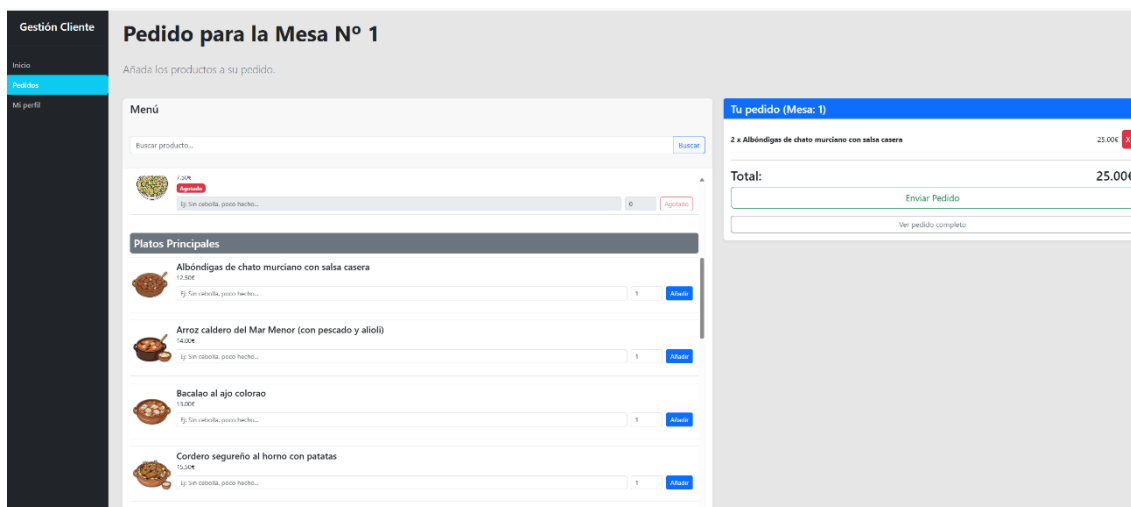
- **Inicio:** En esta página se pueden ver algunas estadísticas propias del cliente como **mesas ocupadas pedidos realizados** y pedidos que ya **han sido servidos**.



- **Reservar mesa:** En esta página se puede reservar mesa, pero con **una salvedad**, y es que cuando ya has reservado mesa, a ese cliente se le asigna **una reserva y un numero de pedido** y no podrá volver a reservar mesa hasta **que cierre su pedido**, de manera que esa pestaña desaparecerá y solo quedaran **tres**, redireccionándose automáticamente a la pestaña de pedido.



Como se puede observar en la imagen directamente sobre la “card” tengo puesto el “input” de tipo “number” que tiene como mínimo un comensal y como máximo deja hasta 12, y un botón que obtiene el número de mesa que asignara al **id del pedido** y el **número de comensales que asignara en la reserva**.



Volviendo a las redirecciones, en el **navbar**, tengo puestas unas **condiciones**, y es que **solo aparezca** la pestaña de reservar mesa **si no tienes ningún pedido activo**.

```
<nav id="sidebar" class="collapsible">
  <div class="sidebar-header">
    <h3>Gestión Cliente</h3>
  </div>
  <ul class="list-unstyled components sidebar-items">
    <li>
      <a href="indexClientes.php"
        class="nav-link <?=( $pagina_actual == 'indexClientes.php') ? 'active' : '' ?>">
        Inicio
      </a>
    </li>
    <?php
      //Solo mostrar el enlace "Reservar Mesa" si NO hay un pedido activo en la sesión
      if (!isset($_SESSION['idped'])) {
      }
      <li>
        <a href="mesas.php"
          class="nav-link <?=( $pagina_actual == 'mesas.php') ? 'active' : '' ?>">
          Reservar Mesa
        </a>
      </li>
    </li>
  </ul>
</nav>
```

Como podemos ver si la **variable de sesión** del pedido **no** ha sido iniciada entonces si tenemos la **pestaña disponible**, pero si **por el contrario** no existiera pues ese bloque **saltaría** al siguiente que es la de los pedidos y en este caso **si habría pedidos**.

- **Pedidos:**

de igual manera que en **reservar mesa**, pero al contrario lo hacemos con la **pestaña de pedidos**, que **no te deje acceder** si no tienes una reserva hecha y por ende un **pedido activo**. De manera que cuando **no tengamos pedido activo** y demos en la pestaña de pedido esa pestaña nos enviara a la página de **pedidos.php** y desde ahí verificara si esta un pedido en activo si no lo está redireccionara hacia **mesas.php** haciendo que no sea accesible, por el contrario, si tienes un **pedido activo** la pestaña de **reservar mesa desaparecerá** y te redireccionara directamente a la pestaña de pedidos que **abrirá tu pedido activo**.

```
// Definimos el enlace por defecto para "Pedidos".
$pedidopendiente = "pedidos.php";

//Comprobamos si existe un 'idped' en la URL actual.
if (isset($_SESSION['idped'])) {
  //Si existe... Sobrescribimos la variable para que el enlace
  // apunte a este pedido específico.
  $pedidopendiente = "pedidos.php?idped=" . (int)$_SESSION['idped'];
}
?>
```

Comenzamos viendo el navbar, el cual **verificamos** que si hay una **variable de sesión activa** de un pedido nos guarde en la variable "**pedidopendiente**" la url que redirigirá directamente a la pagina pedidos, pero con la url del **pedido concreto**.

Aquí podemos observarlo mejor:

```
<li>
  <a href="<?php echo $pedidopendiente; ?>"
    class="nav-link <?=( $pagina_actual == 'pedidos.php') ? 'active' : '' ?>">
    Pedidos
  </a>
</li>
</li>
```

En caso de que la **variable de sesión del pedido no exista** entonces la variable de pedido pendiente valdrá solo la **url de pedidos.php** y en la pagina de pedidos al **no** tener un **pedido activo** te redireccionara a **mesas.php** de esa manera nos aseguramos que el cliente que no tenga un pedido no pueda acceder a esa pestaña por mucho que tenga la url.

Pd: Aunque más arriba no lo detallo claramente **el ternario** que se encuentra en los links es para mantener **la pestaña activa**, o sea que quede **iluminada** cuando te encuentres en esa pestaña. Esto es debido a una variable creada con una función que determina la **posición en la que se encuentra ahora mismo** el usuario **independientemente de la carpeta**.

Que es esta:

```
// Detecta el nombre del archivo actual sin importar la carpeta
$pagina_actual = basename(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH));
```

```
$idped_activo = false;

if (isset($_SESSION['idped'])) {
    $idped = $_SESSION['idped'];
    $idped_activo = true;

    if (isset($_SESSION['idmesa'])) {
        $idmesa = $_SESSION['idmesa'];
    }

    $consulta = "SELECT estado FROM pedido WHERE idped = '$idped'";
    $result = mysqli_query($conn, $consulta);
    $row = mysqli_fetch_array($result);

    // Si el estado es '1' (asumiendo que significa "pedido cerrado/pagado"), redirige
    if ($row && $row['estado'] == '1') {
        unset($_SESSION['idped']);

        mysqli_close($conn);
        header('Location:mesas.php');
        exit();
    }
}

if (!$idped_activo) {
    mysqli_close($conn);
    header('Location:mesas.php'); // Redirige a donde el usuario pueda iniciar un pedido/seleccionar mesa
    exit();
}
```

Y de esta manera resolví ese problema, que si el pedido esta activo o tiene un **estado='1'**, o sea **pagado** pues directamente **redirija a mesas.php** y **borre la variable** de sesión del pedido, para volver a hacer una reserva.

- **Mi perfil:** En esta página básicamente es la edición de nuestro perfil como con los productos y las categorías, utilizo para mantener la **uniformidad y el minimalismo** la función **collapse** de Bootstrap que hace que el formulario en vez de verse de primeras quede **oculto por un botón**

Mi perfil

Gestiona tu información.

[Editar perfil](#)

DNI/NIF

12345678A

Nombre

Antonio

Apellidos

Vicente López

Correo Electrónico

tonikroski@ladespensa.es

Teléfono

858987453

Dirección

C/Murillo,38

Contraseña

Repetir Contraseña

[Aceptar cambios](#)

Lista de facturas

Pedido N°	Cliente	Apellidos	E-mail	Fecha	Importe	Acciones
1	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:29:48	0.00	Ver PDF
2	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:32:25	7.50	Ver PDF
3	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:49:20	32.00	Ver PDF

dejando esa parte más en un **segundo plano** mientras se muestra mejor la parte de nuestro **historial de pedidos**, pudiendo como veremos más adelante verlos **digitalizados en pdf** y con posibilidad de **descargarlos**.

3.2.2 CONTROL DE STOCK

Para el control del stock hice **unos condicionales** a la hora de **añadir** cosas **al carrito**, o sea que cuando das a **añadir al carrito** esa cantidad ya se resta de la base de datos de forma que queda reservada para ese cliente evitando que otro llegue y la pida antes, así a los demás clientes les saldría el **stock disponible** siempre en **tiempo real**, por el contrario, si el cliente decide no pedirlo tan solo lo **devolvería al stock** haciendo otro **update**. Aun así, decidí para que **no haya errores de ningún tipo** meter una **verificación** cuando añado algo al carrito de manera que **captura el error** si no hay stock y la muestra en la pagina de pedidos.php, al igual en la página de pedidos.php también verificamos el stock y si no queda ese producto entonces lo marcara como **agotado**. De esta manera, aunque parezca **mas complejo y redundante** nos aseguramos que **no** se pidan dos productos **simultáneos** y se pueda quedar la base de datos como negativo, cerrando toda posibilidad a que el mas lento de los clientes llegue a pedir porque aun no ha sido actualizada la base de datos.

Muestro unas capturas de lo llevado a cabo mencionado anteriormente, primero comenzamos con la pagina de **añadir al carrito** y luego vemos la **de pedidos**.

```
// vemos si nos llegan los datos o no
if ($_SERVER['REQUEST_METHOD'] == 'POST') {

    $idprod = $_POST['idprod'];
    $cantidad = (int)$_POST['cantidad'];

    // comprobamos el stock disponible
    $consulta_stock = "SELECT * FROM producto WHERE idprod='$idprod'";

    $result = mysqli_query($conn, $consulta_stock);
    $row = mysqli_fetch_assoc($result);
    $stock_disponible = (int)$row['stock'];

    // 2. Verificación previa
    if ($stock_disponible >= $cantidad) {

        // Inicializar carrito
        if (!isset($_SESSION['carrito'])) {
            $_SESSION['carrito'] = array();
        }

        // Guardar producto
        $lista = array(
            'id' => $idprod,
            'nombre' => $_POST['nombre'],
            'precio' => $_POST['precio'],
            'comentario' => $_POST['comentario'],
            'cantidad' => $cantidad
        );

        $_SESSION['carrito'][] = $lista;

        // Descontar automáticamente (Sprint 2)
        $consulta_restar_stock = "UPDATE producto SET stock = stock - $cantidad WHERE idprod = '$idprod'";
        mysqli_query($conn, $consulta_restar_stock);

        mysqli_close($conn);
        // volvemos a la seccion pedidos
        header('Location: pedidos.php');
        exit();
    } else {
        mysqli_close($conn);
        // 3. Mostrar advertencia si no hay stock (Sprint 2)
        header('Location: pedidos.php?error=1');
        exit();
    }
}
```

Observamos que primeramente **se verifica** la llegada de algún dato por parte del **formulario de pedidos**, después con ese producto se **verifica que haya stock**, si lo hay, creamos el carrito y añadimos ese producto al carrito, y después devolvemos a la pagina de pedidos, si por el contrario no hubiera devolvemos **un error** por **\$_GET**.

Ahora en la página de pedidos.php veremos como **capturamos el error** en caso de que exista y como volvemos a **verificar si hay stock en cada producto** antes de enviar el producto de manera que se actualiza el stock también una vez se pulse el botón añadir, dejando un **badge debajo del nombre** avisando del **producto agotado** y el botón de añadir igual que el input **del comentario deshabilitado**.

Aquí tenemos la consulta, aunque en este caso va precedida también del filtrado por búsqueda, pero no extenderemos aun esa información.

```
$consulta_productos = "SELECT idprod, nombre, precio, stock, imagen FROM producto WHERE categoria = '{$idcat}' AND estado = 0";
if ($busqueda != '') {
    $consulta_productos .= " AND nombre LIKE '%$busqueda%'";
}
$consulta_productos .= " ORDER BY nombre ASC";

$result_productos = mysqli_query($conn, $consulta_productos);

// Si hay productos en la categoría (o que coinciden con la búsqueda)
```

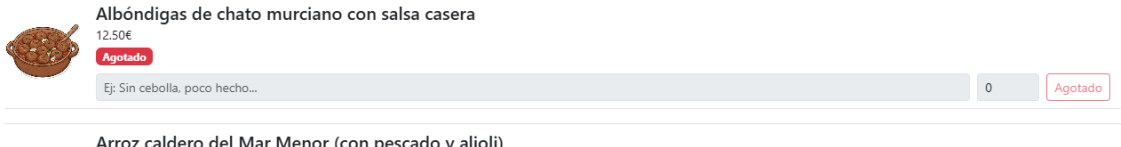
Y como mencionamos anteriormente aquí esta el resto del condicional que verifica si hay stock y mete el badge y el botón deshabilitado

```
<div>
    <h5 class="mb-0"><?php echo $nombre_producto; ?></h5>
    <small><?php echo $precio_producto; ?></small>
</div>
<?php if ($stock_producto <= 0) { ?>
    <span class="badge bg-danger align-self-center">Agotado</span>
<?php } ?>
</div>

<?php if (($stock_producto > 0) && isset($idped)) { // Mostrar formulario solo si hay stock y el pedido existe
?>
<form action="añadir_carro.php" method="POST" class="d-flex flex-wrap gap-2 mt-2">
    <input type="hidden" name="idped" value="<?php echo $idped; ?>">
    <input type="hidden" name="idprod" value="<?php echo $idprod; ?>">
    <input type="hidden" name="nombre" value="<?php echo $nombre_producto; ?>">
    <input type="hidden" name="precio" value="<?php echo $producto['precio']; ?>">

    <div class="flex-grow-1" style="min-width: 150px;">
        <input type="text" name="comentario" class="form-control form-control-sm" placeholder="Ej: Sin cebolla, poco hecho..." maxlength="250">
    </div>

    <div class="d-flex gap-2">
        <input type="number" name="cantidad" value="1" min="1" max="<?php echo $stock_producto; ?>" class="form-control form-control-sm" style="width: 70px;">
        <button type="submit" class="btn btn-primary btn-sm me-3">Añadir</button>
    </div>
</form>
<?php } else { // Bloque que se muestra si NO hay stock disponible o no hay pedido activo
?>
<div class="d-flex flex-wrap gap-2 mt-2">
    <div class="flex-grow-1" style="min-width: 150px;">
        <input type="text" class="form-control form-control-sm" placeholder="Ej: Sin cebolla, poco hecho..." disabled>
    </div>
    <div class="d-flex gap-2">
        <input type="number" value="0" class="form-control form-control-sm" style="width: 70px;" disabled>
        <button type="button" class="btn btn-outline-danger btn-sm me-3" disabled>Agotado</button>
    </div>
</div>
<?php } ?>
```



Albóndigas de chato murciano con salsa casera
12.50€
Agotado
Ej: Sin cebolla, poco hecho... 0 **Agotado**

Arroz caldero del Mar Menor (con pescado y alioli)

3.2.3 IMPLEMENTACION DE NOTAS DE PRODUCTO

Aunque puede que en algunas capturas ya se haya mostrado la opción de poner comentarios al producto, lo detallaremos un poco más, en primer lugar, cabe mencionar que en la tabla de la base de datos de pedido producto se habilita una fila que es la de los comentarios que va asociada al producto y al pedido.

Y en la parte de la pagina de pedidos en la cual vamos a trasladar a la de añadir carrito mediante un formulario post, ahí cogeremos el comentario y lo insertamos en la tabla antes mencionada y la mostramos en el carrito. Un ejemplo grafico de lo que aquí explico seria:

```
<form action="añadir_carro.php" method="POST" class="d-flex flex-wrap gap-2 mt-2">
    <input type="hidden" name="idped" value="<?php echo $idped; ?>">
    <input type="hidden" name="idprod" value="<?php echo $idprod; ?>">
    <input type="hidden" name="nombre" value="<?php echo $nombre_producto; ?>">
    <input type="hidden" name="precio" value="<?php echo $producto['precio']; ?>">

    <div class="flex-grow-1" style="min-width: 150px;">
        <input type="text" name="comentario" class="form-control form-control-sm" placeholder="Ej: Sin cebolla, poco hecho..." maxlength="250">
    </div>

    <div class="d-flex gap-2">
        <input type="number" name="cantidad" value="1" min="1" max="<?php echo $stock_producto; ?>" class="form-control form-control-sm" style="width: 70px;">
        <button type="submit" class="btn btn-primary btn-sm me-3">Añadir</button>
    </div>
</form>
```

Vemos como en la pagina de **pedidos.php** tenemos un input con un **"name"** que sería el de **"comentario"**, el cual recoge la nota del producto añadido y la envía por post a la página **añadir_carro.php**

Tras eso añadimos ese comentario al carrito que será uno de los objetos llamado **comentario**. Y de esa forma **capturamos** ahí en el carrito el comentario, como estos productos ya están en una variable de sesión, lo que hará el botón enviar pedido es procesar el carrito y **añadir o insertar** lo que hay en el a la base de datos (**pedido_producto**) y posteriormente borrar el carrito. Para que se vea un ejemplo de lo que hace el botón de enviar pedido, solo sería leer si el carrito está vacío, y si lo está, que el botón quede **deshabilitado** para que no puedas pulsar y **enviar el carrito vacío**. En resumidas cuentas, lo que hace ese botón es **activar**

la **lógica** que hay detrás de la página **enviar_carro.php**, su funcionamiento ya ha sido explicado unas líneas atrás.

```
<form action="enviar_carro.php" method="POST" class="d-grid">
  <button class="btn btn-outline-success btn-lg" <?php if (empty($_SESSION['carrito'])) echo 'disabled'; ?>>
    <span>Enviar Pedido</span>
  </button>
</form>
```

Este es el funcionamiento del **botón de enviar pedido** y ahora veremos la lógica de hacia donde va o que hace esa página.

```
// comprobacion del carrito no vacio
if (!empty($_SESSION['carrito'])) {

    // creamos una variable a partir de la sesion del dni
    $dni = $_SESSION['dni'];

    // Obtenemos el id del pedido
    $consulta_idpedido = "SELECT * FROM pedido WHERE usuario='$dni' AND estado=0";
    $result = mysqli_query($conn, $consulta_idpedido);

    $row = mysqli_fetch_assoc($result);
    $idped = $row['idped'];

    // variable de sesión del idped
    $_SESSION['idped'] = $idped;

    // Recorremos el carrito
    foreach ($_SESSION['carrito'] as $producto) {

        // productos del carrito tabla pedido_producto
        $idprod = $producto['id'];
        $comentario = $producto['comentario'];
        $cantidad = $producto['cantidad'];
        $consulta_producto = "INSERT INTO pedido_producto (idped, idprod, cant,comentario, servido)
        VALUES ('$idped', '$idprod','$cantidad', '$comentario', 0)";

        mysqli_query($conn, $consulta_producto);
    }
    mysqli_close($conn);

    ✦ unset($_SESSION['carrito']);

    // header('Location:pedidos.php?idp=' . $idped);
    // exit();
    header('Location: tickets/generar_ticket.php?idp=' . $idped);
    exit();
} else {

    // aseguramos entrada sin sesion o sin productos
    header('Location:pedidos.php');
    exit();
}
```

Como explicamos **anteriormente** como ese carrito ya ha sido creado y está en la **sesión** lo único que hace esta pagina es hacer un **insert** con los datos del carrito e insertar en la tabla **pedido_producto** para después desestear el propio carrito (**limpiar la variable**) ya que en cuanto vuelvas a añadir otro producto como se vio anteriormente se vuelve a crear con los nuevos productos y así sucesivamente. **Pd: tengo comentada una redirección a pedidos.php para hacer las pruebas sin que me diera error por no tener impresora activa.**

3.2.4 BUSCADOR DE PRODUCTOS

Aunque se pasó por encima un poco por esta funcionalidad la veremos en este punto un poco más detenidamente, en primer lugar, vemos como creamos la variable para que al darle al botón de buscar lea lo que hemos introducido para meterlo a la consulta y que nos muestre el resultado o nos lo imprima.

En primera diapositiva tenemos el botón de buscar el cual obtiene el valor del input y lo devuelve a la propia página por "GET"

```
<form method="GET" class="mb-4">
  <div class="input-group">
    <input type="text" name="buscar" class="form-control" placeholder="Buscar producto...">
    <button class="btn btn-outline-primary" type="submit">Buscar</button>
  </div>
</form>
```

en esta segunda foto tenemos la **captura de la variable** por "GET" y la convertimos en una variable global para meterla a la **consulta cuando se genera la carta**.

```
// 1. Obtener el término de búsqueda
if (isset($_GET['buscar'])) {
    $busqueda = $_GET['buscar'];
} else {
    $busqueda = '';
}
```

aquí vemos como hacemos **una consulta al producto para ir imprimiendo mas abajo** aunque no se ve aun, todos los productos de cada categoría, nos centramos en que **cogemos el resultado de la búsqueda** y hacemos el "SELECT" con la **búsqueda** que hemos hecho de manera que **solo nos mostrara los productos que coincidan con esa búsqueda** y que estén **disponibles**, en la imagen muestra que si la variable búsqueda **no esta vacía** entonces coge el contenido y lo **añade a la consulta** buscando por nombre el producto según lo filtrado, sin embargo **si esta vacío** pues simplemente sigue el flujo y mostrara todo sin excepción en orden de aparición según el **nombre del producto**.

```
$consulta_productos = "SELECT idprod, nombre, precio, stock, imagen FROM producto WHERE categoria = '$idcat' AND estado = 0";
if ($busqueda != '') {
    $consulta_productos .= " AND nombre LIKE '%$busqueda%'";
}
$consulta_productos .= " ORDER BY nombre ASC";
```

Menú

michirones

Buscar

Entrantes



Michirones (habas secas guisadas con chorizo y pan)

6.50€

Agotado

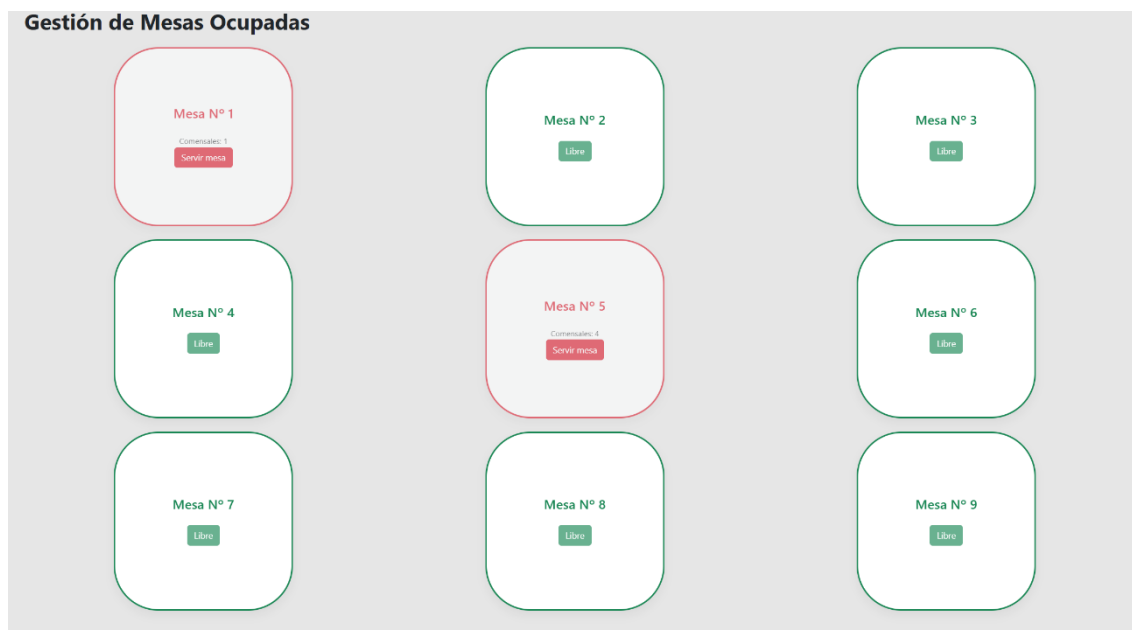
Ej: Sin cebolla, poco hecho...

0

Agotado

3.2.5 MARCADO DE PRODUCTOS VISTA CAMARERO

En esta sección detallamos las **vistas del camarero** y veremos la **gestión de las mesas** y la información **llegada desde el cliente**, básicamente hace una lectura de **las mesas ocupadas y el pedido asociado** a esa mesa, hace una **lectura de los productos que no están servidos** y los muestra con la finalidad de que, al darle al **botón servir**, cambiarle el estado a **servido** y que al cliente le salga como **servido**, añadí la condición de que **si no están todos servidos el botón no deje pagar la cuenta o generar ticket**, en este caso son ambas.



En el caso del camarero incluí la misma vista que la del cliente cuando reserva mesa para mayor **homogeneidad visual** y que no fuera mediante **desplegables o tablas**.

Al pulsar sobre uno de ellos vemos que pedidos de que cliente hemos **servido y cuáles no**.

En esta imagen vemos **representativamente** como si no tenemos todos los ítems servidos el botón queda **deshabilitado** y **no nos dejaría cerrar la mesa**, eso lo hice **para un mayor control y seguridad sobre el stock**, puesto que si pagara la mesa con **productos sin servir** no se añadirían a la cuenta y por tanto quedaría ese producto en el limbo, **es decir, fuera de stock y encima sin cobrar**.

Resumen del Pedido de la Mesa '1'

Producto	Comentarios	Cantidad	Estado	Acción
Albóndigas de chato murciano con salsa casera	2	2	PENDIENTE	SERVIR
Arroz caldero del Mar Menor (con pescado y alioli)	ASDF	2	SERVIDO	SERVIDO
Arroz caldero del Mar Menor (con pescado y alioli)		86	PENDIENTE	SERVIR

Generar Cuenta

El cliente ha pedido la cuenta. Púlsalo aquí para imprimir el ticket.

GENERAR TICKET Y CERRAR MESA

— VOLVER

Resumen del Pedido de la Mesa 1

Total Pedido a la Mesa

Producto	Cantidad	Comentario	Estado	Precio
Albóndigas de chato murciano con salsa casera	2	2	Pendiente	12.50 €
Arroz caldero del Mar Menor (con pescado y alioli)	2	ASDF	Servido	14.00 €
Arroz caldero del Mar Menor (con pescado y alioli)	86		Pendiente	14.00 €
Total a Pagar:				1,257.00 €

Así se vería desde el lado del cliente los **botones de servir por parte del camarero** quedan **deshabilitados** también una vez que el camarero le da a servir, **cambiando de estado y también su color**.

Vamos a ver un poco mas a fondo la **lógica que entraña el servir el pedido**.

```
<form method='POST' style='display:inline;'>
  <input type='hidden' name='id_linea' value='<?php echo $producto['idline'];?>'>
  <?php if (!$esta_servido) {?>
    <button type='submit' name='marcar_servido' class='btn btn-sm btn-outline-success'>SERVIR</button>
  <?php }else{ ?>
    <button class='btn btn-sm btn-outline-secondary' disabled>SERVIDO</button>
  <?php } ?>
</form>
```

Aquí tenemos la lógica del **botón de servir el producto**, el cual al hacer un redireccionamiento sobre la propia pagina no hace falta poner el **"action"** y solo el **método** que en este caso será el **"POST"**, como vemos al hacer el post metemos un **input oculto** para mover la información y que no se borre al recargar la página, con la información de la línea que sirve, la cual es **una variable que crea** para luego manejar, al igual que la variable de **marcar_servido**, ambas las vemos mas arriba en el principio de la pagina y vemos que hace un update sobre la tabla de pedido_producto.

```
// Marcamos el producto como servido
if (isset($_POST['marcar_servido'])) {

    $id_linea = $_POST['id_linea'];
    //actualizamos cada vez que sirvamos un producto
    $actualizar_servido = "UPDATE pedido_producto SET servido = 1 WHERE idline = $id_linea";
    mysqli_query($conn, $actualizar_servido);
    //y redirigimos a la misma pagina para que se vea el cambio actualizado
    header("Location:servir_pedido.php?id=".$id_mesa);
    exit();
}
```

Vemos como recibimos los datos de **id_linea** por **post** como indicaba en la imagen anterior y actualiza el estado del producto a **servido** y redirige a la misma pagina con el **id de la mesa** para que se actualice la tabla.

```
// Buscamos si hay un pedido activo en esta mesa
$consulta_pedido = "SELECT * FROM pedido WHERE nmesa = $id_mesa AND estado = 0";
$resultado_pedido = mysqli_query($conn, $consulta_pedido);
$pedido = mysqli_fetch_assoc($resultado_pedido);

// Comprobamos si hay productos pendientes de servir
$productos_pendientes = false;

if ($pedido) {
    $idped = $pedido['idped'];
    $sql_pendientes = "SELECT * FROM pedido_producto WHERE idped = $idped AND servido = 0";
    $res_pendientes = mysqli_query($conn, $sql_pendientes);
    if (mysqli_num_rows($res_pendientes) > 0) {
        $productos_pendientes = true;
    }
}
```

Por ultimo vamos a ver la **lógica del booleano** de **pedidos pendientes** la cual viene de una consulta simplemente

aquí hacemos una **consulta buscando un pedido activo** para la mesa activa en este momento y **creamos e inicializamos al variable a false**, después vemos **con un condicional que si pedido tiene algún resultado** entonces buscaremos algún producto que **no esté servido** y si tiene algún resultado entonces **habrá pedidos pendientes**, si no la variable seguirá siendo false y por tanto el **botón de pagar se desbloqueara**.

3.3-SPRINT 3

3.3.1 GENERACION DE CUENTAS Y CIERRE DE MESAS

Para el **cierre de la mesa**, mi lógica es más **sencilla**, puesto que como hemos visto anteriormente casi todo el código que lleva la página **servir_pedido.php**, ahí también va el botón de **generar ticket y cierre de mesa** todo en uno, tome la decisión de que sirvas todos los productos y puedas volver hacia atrás a servir otras mesas de manera que el cliente **pueda seguir haciendo rondas y rondas** hasta que pida la cuenta y en se momento ya le darías a cerrar mesa y te **generaría el ticket y automáticamente cierra la mesa**, **impidiendo** que el usuario **vuelva a pedir** y poniendo en las bases de datos **de nuevo todo a 0 (disponible)**.

Vamos a ir explicando primero el **botón de cerrar mesa** ya que hace referencia a la propia página y entonces no hace falta un redireccionamiento **solo que pongamos el método por el que va a enviar la información**.

```
<!-- Generar cuenta -->
<div class="col-md-4">
    <div class="card">
        <div class="card-header bg-primary text-white"><h4>Generar Cuenta</h4></div>
        <div class="card-body text-center">
            <p>El cliente ha pedido la cuenta. Pulsa aquí para imprimir el ticket.</p>
            <form method="POST">
                <input type="hidden" name="idped" value="<?php echo $pedido['idped'] ?? 0; ?>">
                <button type="submit" name="generar_cuenta" class="btn btn-outline-success btn-lg w-100">
                    <?php if ($productos_pendientes) echo 'disabled'; ?>>
                    GENERAR TICKET Y CERRAR MESA
                </button>
            </form>
        </div>
    </div>
</div>
</div>
</div>
```

Como vemos en el botón utilizo la **misma dinámica que en el otro botón del cliente**, básicamente un condicional con la variable, la cual es un **flag** que si hay algún pedido pendiente pues quedaría deshabilitado. Ahora en las siguientes capturas veremos cuales son las fases que sigue esta página.

En primera instancia ese botón **envía por post** la variable **generar_cuenta** la cual nos servirá para explicar un trozo en las siguientes diapositivas.

antes que nada, vemos la **instalación de la última versión** de la impresora.

```
./composer.json has been created
Running composer update mike42/escpos-php
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking mike42/escpos-php (v4.0)
  - Locking mike42/gfx-php (v0.6)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Installing mike42/gfx-php (v0.6): Extracting archive
  - Installing mike42/escpos-php (v4.0): Extracting archive
1 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
No security vulnerability advisories found.
Using version ^4.0 for mike42/escpos-php
```

Aquí tenemos el **fragmento de código mencionado anteriormente** el cual hace que, una vez dado al botón de generar cuenta, hace **tres updates** uno para **liberar la mesa** otro para marcar la **reserva** como **terminada**, y otro para que el estado del **pedido sea pagado**.

Después del cierre de mesas te **redirige** hacia la página de **generación de tickets** enviando por **\$_GET** las **dos variables**, la de **identificador de la mesa** y el **identificador del pedido** para poder hacer en dicha página las consultas pertinentes, y de ahí te **generaría el ticket** y ya te devuelve a la página de gestionar mesas.

```
// Generamos la cuenta (imprimir ticket)
if (isset($_POST['generar_cuenta'])) {

    $idped = $_POST['idped'];

    // Liberamos la mesa
    mysqli_query($conn, "UPDATE mesa SET estado = 0 WHERE nmesa = $id_mesa");
    // Marcamos la reserva como terminada
    mysqli_query($conn, "UPDATE reserva SET estado = 1 WHERE nmesa = $id_mesa");
    // Marcamos el pedido como pagado
    mysqli_query($conn, "UPDATE pedido SET estado = 1 WHERE nmesa = $id_mesa AND estado = 0");

    // header("Location:servir_pedido.php?id=".$id_mesa."&idp=".$idped);
    // exit();

    header("Location:tickets/generar_ticket.php?idm=" . $id_mesa . "&idp=" . $idped);
    exit();
}
```

Adjunto una **captura** de como seria la pagina de los **tickets de las cuentas** y luego iremos a mencionar la otra página de los **tickets de la cocina** ya que forma parte del mismo proceso prácticamente, aunque ese ticket se detallara **más adelante**.

En este fragmento vemos como obtenemos las variables que venían del **GET** que envió **servir_pedido.php**,

```
if (isset($_GET['idp'])) {

    // Variables
    $idped = $_GET['idp'];
    $mesa_id = $_GET['idm'];
    $precio_final = 0;
```

Y aquí la configuración de dicha impresora desde el archivo de **generar_tickets.php**

```
// Configurar impresora - Usar conexión de red
$ipImpresora = "192.168.36.170"; // Cambiar a la IP de tu impresora
$puertoImpresora = 9100; // Puerto por defecto para impresoras ESC/POS
$connector = new NetworkPrintConnector($ipImpresora, $puertoImpresora);
$printer = new Printer($connector);
```

Para no hacer muy extenso el documento iré directamente a la consulta y donde insertaremos el contenido en el ticket, después una foto del ticket de cuenta.

```
// Consulta pedido
$query = "SELECT
    p.nombre,
    SUM(pp.cant) AS cantidad,
    p.precio,
    SUM(pp.cant * p.precio) AS total
FROM pedido_producto pp, producto p
WHERE pp.idprod = p.idprod
AND pp.idped = '$idped'
GROUP BY p.nombre, p.precio";
$result = mysqli_query($conn, $query);
while ($row = mysqli_fetch_array($result)) {

    $cantidad = $row['cantidad'];
    $precio = $row['precio'];

    //asignamos un corte en el nombre del producto para poder tener una buena legibilidad en el ticket
    $nombre = substr($row['nombre'], 0, 35);

    $printer->text(sprintf("%40s %3s %5s %s\n", $nombre, str_pad($cantidad, 3, ' ', STR_PAD_BOTH), str_pad(number_format($precio, 2), 5, ' ', STR_PAD_BOTH), number_format($row['total'], 2)));

    $precio_final += $row['total'];
}

// Cierro conexión
mysqli_close($conn);

// calculamos la Base imponible y el iva
$baseimponible = $precio_final / 1.21;
$iva = $baseimponible * 0.21;

// Totales
$printer->text(str_repeat("-", 58) . "\n");
$printer->setJustification(Printer::JUSTIFY_RIGHT);
$printer->text(sprintf("%41s %13.2f EUR\n", "Base imponible: ", $baseimponible));
$printer->text(sprintf("%41s %13.2f EUR\n", "IVA (21%): ", $iva));
$printer->text(str_repeat("-", 58) . "\n");
$printer->setEmphasis(true);
$printer->text(sprintf("%41s %13.2f EUR\n", "TOTAL: ", $precio_final));
$printer->setEmphasis(false);
```

Aunque no se vea de forma perfecta al menos podemos **ver la consulta** que se hace, la cual se hace a **dos tablas diferentes**, para obtener **nombre del producto**, la **cantidad de ese producto**, el **precio del producto** y el **importe del producto**, el cual es fruto del sumatorio de la multiplicación de la cantidad de productos por el precio de este y te saldrá el total.

Después de la consulta hacemos un **bucle while** para ir **imprimiendo línea a línea todos los productos con sus cantidades y su importe**. Aunque adjunto una foto del ticket real, durante la formación de este documento fui moldeando algo mas y lo perfeccione un poco, el cual ahora como vemos el código no solo tenemos el nombre y la cantidad si no el precio unitario y luego el importe, como vemos en la cantidad y en el total utilizo la **función STR_PAD_BOTH** que lo que hace es **centrar el contenido en la columna**. Indicando la variable, el número de caracteres que consume y el tipo de alineado. Luego mas abajo vemos el calculo de la base imponible y ya la inserción de los datos de la cuenta total.

Adjunto una foto del ticket de cuenta.



3.3.2 FACTURAS DE CLIENTE

En este apartado veremos la **generación de pdf** en el **área de cliente**, en el perfil de cliente saldrán las **facturas de cada uno de ellos**, veremos que nos muestra el pedido del cliente de una reserva en una factura en formato pdf.

Para comenzar vamos con la instalación, igual que con la impresora, pero esta vez con **mPDF** para ello **instalaremos la última versión a través de composer**.

```
./composer.json has been updated
Running composer update mpdf/mpdf
Loading composer repositories with package information
Updating dependencies
Lock file operations: 8 installs, 0 updates, 0 removals
- Locking mpdf/mpdf (v8.2.6)
- Locking mpdf/psr-http-message-shim (v2.0.1)
- Locking mpdf/psr-log-aware-trait (v3.0.0)
- Locking myclabs/deep-copy (1.13.4)
- Locking paragonie/random_compat (v9.99.100)
- Locking psr/http-message (2.0)
- Locking psr/log (3.0.2)
- Locking setasign/fpdi (v2.6.4)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 8 installs, 0 updates, 0 removals
- Installing setasign/fpdi (v2.6.4): Extracting archive
- Installing psr/log (3.0.2): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing paragonie/random_compat (v9.99.100): Extracting archive
- Installing myclabs/deep-copy (1.13.4): Extracting archive
- Installing mpdf/psr-log-aware-trait (v3.0.0): Extracting archive
- Installing mpdf/psr-http-message-shim (v2.0.1): Extracting archive
- Installing mpdf/mpdf (v8.2.6): Extracting archive
2 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
3 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^8.2 for mpdf/mpdf
```

Una vez realizada la instalación vamos al archivo en cuestión, antes de eso vamos a ver otra vez en el perfil del cliente y vemos que nos salen todos los pedidos realizados por ese cliente con las **fechas de inicio de la reserva o pedido**.

Pedido N°	Cliente	Apellidos	E-mail	Fecha	Importe	Acciones
1	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:29:48	0.00	Ver PDF
2	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:32:25	7.50	Ver PDF
3	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 10:48:20	32.00	Ver PDF
4	Antonio	Vicente López	tonikroski@ladespensa.es	2025-12-24 13:27:01	1,283.00	Ver PDF



Aunque el diseño **no queda totalmente estético** porque el logo **se ve con fondo**, no logre hacer que **mPDF aceptara transparencias como png**. Y decidí dejarle ese fondo como de papel reciclado por la estética huertana y que parezca más acorde con la temática del restaurante.

Ahora veremos la lógica detrás de este apartado. Empezamos como siempre por el **detonante** que es el **botón de ver pdf**,

```
<tr class='text-center'>
  <td> <?php echo $idped; ?> </td>
  <td> <?php echo $nombre; ?> </td>
  <td> <?php echo $apellidos; ?> </td>
  <td> <?php echo $email; ?> </td>
  <td> <?php echo $fecha; ?> </td>
  <td> <?php echo number_format($total, 2); ?> </td>
  <td> <a href="PDF/factura.php?id= ' <?php echo $idped; ?> '" class="btn btn-sm btn-outline-success">Ver PDF</a></td>
</tr>
```

Aquí vemos que cogemos el **id del pedido** y lo enviamos por **GET** a **factura.php** y lo recogemos como vemos en la siguiente imagen

```
// Verificamos que llegue el ID del pedido
if (isset($_GET['id'])) {
    $idped = $_GET['id'];

    $dni = $_SESSION['dni'];
```

y de ahí sacamos la **fecha del pedido** y los **datos del cliente**, con la **variable de sesión** del cliente.

```
$dni = $_SESSION['dni'];
$precio_final = 0;

// ===== CONSULTA 1: Fecha del pedido =====
$consulta_fecha = "SELECT * FROM pedido WHERE idped = $idped";
$result1 = mysqli_query($conn, $consulta_fecha);

$row1 = mysqli_fetch_array($result1);

$fecha = $row1['fecha'];

// ===== CONSULTA 2: Datos del cliente =====
$consulta_cliente = "SELECT * FROM usuario WHERE dni = '$dni'";
$result3 = mysqli_query($conn, $consulta_cliente);

$row3 = mysqli_fetch_array($result3);

$nombre = $row3['nombre'];
$apellidos = $row3['apellidos'];
```

Y ya estaría el resto del ticket, pero lo que viene siguiente es la **estructura** y la **inserción de las variables** que hemos obtenido en ella.

```
// ===== ESTRUCTURA HTML DEL TICKET =====
$html = "

<style>
    body {
        background-image: url('../img/fondopdf.jpg');
        font-family: Arial, sans-serif;
    }
</style>
<p align='center'><img src='\" . $url_base . \"img/LaDespensalogo.png' alt='Logo' width='70px'></p>
<h1 align='center'>RESTAURANTE LA DESPENSA</h1>
<br>
<p align='right'><b>Fecha y hora:</b> \" . $fecha . \"</p>
<b>DNI:</b> \" . $dni . \" <br>
<b>Nombre:</b> \" . $nombre . \" <br>
<b>Apellidos:</b> \" . $apellidos . \" <br><br>
<hr>
<table border='none' cellpadding='5' cellspacing='0' width='100%'>
    <thead>
        <tr style='background: RGB(230, 255, 230,);'>
            <th style='padding-bottom: 10px; padding-top: 10px; align='left'>Producto</th>
            <th style='padding-bottom: 10px; padding-top: 10px; align='center'>Cantidad</th>
            <th style='padding-bottom: 10px; padding-top: 10px; align='right'>Precio Unidad</th>
            <th style='padding-bottom: 10px; padding-top: 10px; align='right'>P. Total</th>
        </tr>
    </thead>
    <tbody>
";

$consulta_pedido = "SELECT
    p.nombre,
    SUM(pp.cant) AS cantidad,
    p.precio,
    SUM(pp.cant * p.precio) AS total
FROM pedido_producto pp
INNER JOIN producto p ON pp.idprod = p.idprod
WHERE pp.idped = $idped
GROUP BY p.nombre, p.precio";

$result2 = mysqli_query($conn, $consulta_pedido);
```

```
// Si no hay productos
if (mysqli_num_rows($result2) == 0) {
    $html .= "<tr><td colspan='4' align='center'>No hay productos en este pedido</td></tr>";
} else {
    // Recorremos cada producto
    while ($row2 = mysqli_fetch_array($result2)) {
        $html .= "<tr>
            <td style='padding-top: 10px;'>\" . $row2['nombre'] . \"</td>
            <td style='padding-top: 10px; align='center'>\" . $row2['cantidad'] . \"</td>
            <td style='padding-top: 10px; align='right'>\" . number_format($row2['precio'], 2) . \"</td>
            <td style='padding-top: 10px; align='right'>\" . number_format($row2['total'], 2) . \"</td>
        </tr>";

        // Sumamos al precio total
        $precio_final += $row2['total'];
    }

    // Cerramos la conexión
    mysqli_close($conn);

    $baseimponible=$precio_final/1.21;
    $iva=$baseimponible*0.21;

    // Cerramos la tabla y añadimos el total con el iva y el total
    $html .= "
        </tbody>
    </table>
    <hr>
    <p align='right'><b>Base Imponible: \" . number_format($baseimponible, 2) . \"</b></p>
    <p align='right'><b>IVA(21%): \" . number_format($iva, 2) . \"</b></p>
    <p align='right'><b>Total: \" . number_format($precio_final, 2) . \"</b></p>
    <br>
    <p align='center'><b>¡Gracias por su visita!</b></p>
    ";

    // Generamos el PDF
    $mpdf->WriteHTML($html);
    $mpdf->Output();
}
```

3.4-SPRINT 4

3.4.1 SERVICIOS DE ENVIO A COCINA Y TICKETS

como hemos desglosado anteriormente y con el ticket de la cuenta vamos a ver también el de cocina, el cual ya lo hemos visto más arriba sale de servir pedido hacia **generar_ticket**, adjunto una foto del ticket de cocina y ya vemos el **código del ticket**



Desglosando un poco el código del ticket, vemos también la **redirección** al pedido de nuevo para que el cliente pueda **seguir pidiendo sin salir de la página**.

Vemos de dónde vienen las variables y las seteamos, después viene la **conexión** con la impresora

```
if (isset($_GET['idp'])) {

    // =====
    // VARIABLES
    // =====
    $idped  = $_GET['idp'];
    $mesa_id = $_SESSION['idmesa'];

    // =====
    // CONEXIÓN IMPRESORA
    // =====
    $ipImpresora    = "192.168.36.170";
    $puertoImpresora = 9100;

    $connector = new NetworkPrintConnector($ipImpresora, $puertoImpresora);
    $printer    = new Printer($connector);

    // =====
}
```

Ahora vamos directamente a las consultas

```
// =====
// PRODUCTOS - ULTIMOS AÑADIDOS
// =====
$sql_productos = "
    SELECT p.nombre, pp.comentario, pp.cant
    FROM pedido_producto pp
    INNER JOIN producto p ON pp.idprod = p.idprod
    WHERE pp.idped = $idped AND pp.servido=0
    ORDER BY pp.idline ASC
";

$resultado_productos = mysqli_query($conn, $sql_productos);

if ($resultado_productos && mysqli_num_rows($resultado_productos) > 0) {
    while ($producto = mysqli_fetch_assoc($resultado_productos)) {
        $nombre      = substr($producto['nombre'], 0, 29);
        $comentario   = substr($producto['comentario'], 0, 15);
        $cantidad     = (int)$producto['cant'] ?? 1;

        // Imprimimos una línea por unidad (ideal para cocina)
        $printer->text(sprintf("%-30s %6s %19s\n", $nombre, str_pad($cantidad, 6, ' ', STR_PAD_BOTH), $comentario));
    }
}
```

Como vemos en la imagen hacemos una consulta en la cual obtenemos a través de **id del pedido** una consulta compuesta por **dos tablas** y de ahí sale el **nombre del producto, el comentario y la cantidad que serán suficientes datos para el ticket**, como vemos aquí hacemos una consulta a la base de datos pero también cabe la posibilidad de hacer un **bucle foreach** de la variable **"carrito"** e ir recorriéndolo para ir obteniendo las mismas variables, de esta forma **solo obtendríamos lo que mandamos al carrito**, o sea, lo que **contuviera** el carrito en ese último pedido, pero de la manera que lo he hecho yo, con **consulta compuesta**, lo que hace es imprimir los tickets de los pedidos que **no estén servidos** de manera que en la cocina nunca se olvidaran de los pedidos que no estén servidos aunque les sigan saliendo comandos nuevas.

por último, vemos que **se limpia el carrito de productos** haciendo que lo que hayas pedido **no se quede ni se reserve** ningún artículo en tu carrito. Y para terminar se **redirecciona** a pedidos, pero con **ese id** de pedido.

```
// }
} else {
    $printer->text("NO HAY PRODUCTOS EN EL PEDIDO\n");
}

$printer->text(str_repeat("-", 56) . "\n\n");

// =====
// PIE DEL TICKET
// =====

$printer->setJustification(Printer::JUSTIFY_CENTER);
$printer->text("Conservar ticket hasta\n");
$printer->text("fin de servicio\n\n");

// =====
// CORTE Y CIERRE
// =====

$printer->cut();
$printer->close();

// =====
// LIMPIAR CARRITO
// =====
unset($_SESSION['carrito']);

// Redirigimos al pedido
header("LOCATION: ../pedidos.php?id='$idped'");
exit();

mysqli_close($conn);

(Exception $e) {
    echo "Error al imprimir ticket: " . $e->getMessage();
}
```

3.4.2 CONTROL DE USUARIOS REGISTRO Y BLOQUEO

En la **Gestión de personal o usuarios**, se encarga de añadir o bloquear usuarios, encargado o camarero

Gestión de Usuarios

Administra diferentes perfiles.

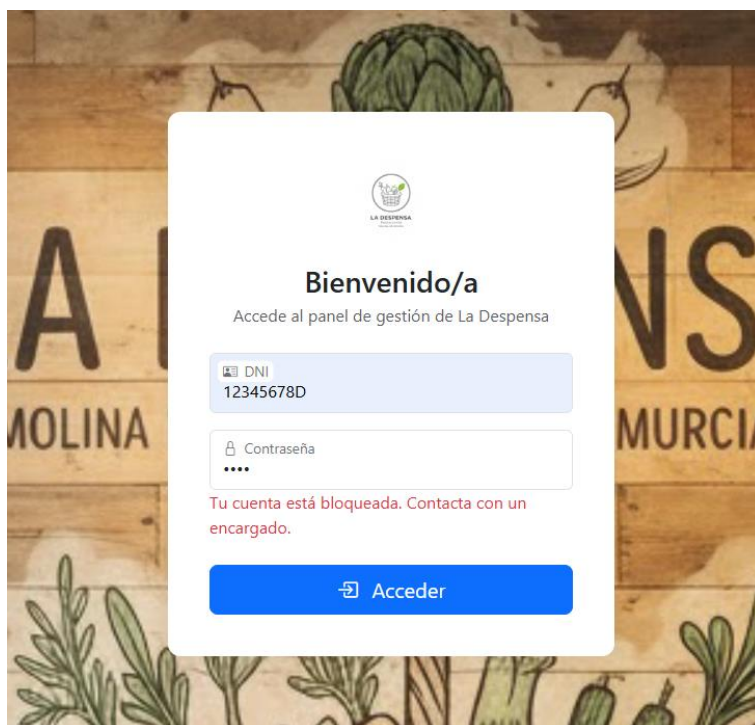
[Añadir nuevo usuario](#)

DN/NIF Ej: 12345678X	Nombre Ej: Juan Pérez	Apellidos Ej: López Otrador
Correo Electrónico ejemplo@ladespensa.com	Teléfono Ej: 628986134	Dirección Ej: C/Gutiérrez Mellado, 12
Rol en el Restaurante Selecciona un rol	Contraseña	Repetir Contraseña

[Registrar Nuevo Usuario](#)

dni	Nombre	Apellidos	email	telefono	Rol	Acciones
12345678A	Antonio	Vicente López	tonikroski@ladespensa.es	658967453	Cliente	Bloquear
12345678B	Alvaro	Martínez Sanz	alvarodevs@ladespensa.es	658965913	Camarero	Bloquear
12345678C	Adrián	Vicente López	adrianvincent@ladespensa.es	685248569	Encargado	Bloquear
12345678D	Julian	Vicente Lopez	adrianvincent17@gmail.com	654789321	Cliente	Desbloquear

En la parte de añadir usuarios decidí hacerla como un **collapse** para obtener más **minimalismo y más énfasis** en el listado de usuarios, el cual salen todos los usuarios **registrados en la web** indistintamente de su rol, y desde ese panel se puede **bloquear o desbloquear** al usuario en cuestión.



Para ese **error**, decidí hacerlo como una **variable de sesión del propio error** en el login. En la captura del **login se verifica que esta o no el usuario bloqueado**, y si lo está, capturaría el error y lo guarda en la variable mostrándolo en el **index**.

```
//Comprobamos que el usuario este bloqueado
if ($row['estado'] == 1) {
    $_SESSION['error_login'] = "Tu cuenta está bloqueada. Contacta con un encargado.";
    header("Location:index.php");
    exit();
}
```

Aquí se ve el error una vez comprobado si el **estado del usuario que ha intentado ingresar es bloqueado**, si lo es, guarda el error y **redirecciona al index**.

Y en el index, vemos que si esa **variable esta iniciada**, entonces la mostrara debajo del **input de la contraseña** y después la **borrara de nuevo**.

```
<div class="form-floating mb-4">
    <input type="password" class="form-control" id="pass" name="pass"
        placeholder="Contraseña" required>
    <label for="pass"><i class="bi bi-lock me-2"></i>Contraseña</label>

    <?php
    if (isset($_SESSION['error_login'])) {
        echo "<p class='text-danger'" . $_SESSION['error_login'] . "</p>";
        unset($_SESSION['error_login']);
    }
    ?>
</div>
```

3.4.2 GENERACION DE INFORMES ESTADISTICOS

Aunque en **cada index de cada rol de usuario tengo unos pequeños bloques estadísticos**, es más mera información temporal del día, y por tanto es en el apartado de encargado que existe, la pestaña estadística, que muestra los números de comensales y los ingresos por días desde una **fecha inicial** hasta una **fecha final**.

Como **cliente** tenemos estos datos.

Mesas Ocupadas 2 / 9 Mesas atendidas actualmente	Pedidos Pendientes 0 Enviados a cocina	Pedidos Servidos 4 Pedidos ya servidos
---	---	---

Como **camarero**, estos datos.

Mesas Ocupadas 1 / 9 Mesas atendidas actualmente	Pedidos Pendientes 1 Enviados a cocina	Pedidos Servidos 8 Pedidos ya servidos
---	---	---

y por último como **encargado** puse estos datos.

Mesas Ocupadas 1 / 9 Mesas atendidas actualmente	Total Transacciones 2 Pedidos terminados	Total Recaudación 1,373.50€ Caja total de hoy
---	---	--

Ahora mostramos la captura de la sección **estadísticas.php** del encargado



Restaurante La Despensa

Bienvenido/a, Adrián [Cerrar Sesión](#)

Gestión Restaurante

- Inicio
- Gestión de Personal
- Productos
- Categorías
- Estadísticas**

Estadísticas del negocio

Fecha Inicio: Fecha Fin:

[Generar estadísticas](#)

Estadísticas

Fecha	Total Comensales	Total Ingresos
-------	------------------	----------------

Como vemos en la imagen va de una **fecha hasta otra** y los desglosa por días, como total de comensales ese día y total de ingresos de todos los pedidos de ese día.

Estadísticas del negocio

Fecha Inicio

18/12/2025

Fecha Fin

28/12/2025

Generar estadísticas

Estadísticas

Fecha	Total Comensales	Total Ingresos
2025-12-24	8	39.50 €
2025-12-28	5	1,373.50 €

```

if (isset($_SESSION['fechainicial'])) {
    $fechainicial = $_SESSION['fechainicial'];
    $fechafinal = $_SESSION['fechafinal'];
}else{
    $fechainicial = '';
    $fechafinal = '';
}

?>

<!DOCTYPE html>
<html lang="es">

<head>
    <?php
    include("../head.php");
    ?>
    <title>Estadísticas - Restaurante La Despensa</title>
</head>

<body class="d-flex flex-column min-vh-100">
    <?php include("../nav.php"); ?>

    <div class="wrapper">
        <?php include("navbar.php"); ?>

        <main class="container mt-4 card">
            <h1>Estadísticas del negocio</h1>

            <form action="logica_estadisticas.php" method="POST" class="row g-3 mb-4">
                <div class="col-md-6">
                    <label for="fechainicial" class="form-label">Fecha Inicio</label>
                    <input type="date" id="fechainicial" name="fechainicial" class="form-control" value="<?php echo $fechainicial; ?>" required />
                </div>
                <div class="col-md-6">
                    <label for="fechafinal" class="form-label">Fecha Fin</label>
                    <input type="date" id="fechafinal" name="fechafinal" class="form-control" value="<?php echo $fechafinal; ?>" required />
                </div>
                <div class="col-12">
                    <button type="submit" class="btn btn-outline-success">Generar estadísticas</button>
                </div>
            </form>
        </main>
    </div>

```

Como vemos en el código de estadísticas, creamos **dos variables de sesión para fecha inicial y fecha final**, que será las que mandemos a la **lógica_estadisticas.php** para calcular el baremo de fechas y desglosarlo por días. también hacemos un recorrido por una variable de **sesión creada como estadísticas que en realidad es un array de objetos**, el cual se compone de **fecha, comensales e ingresos**. De manera que ira recorriendo ese array e imprimiendo día por día el total de comensales, e ingresos, así como la fecha de ese día desde el baremo marcado.

Ahora veremos el propio array y su recorrido mediante un **bucle foreach** igual que el del carrito, pero esta vez con los **datos de las estadísticas**.

Como después veremos la parte de lógica ahora nos centramos en **explicar esta parte del bucle**, el cual nos muestra que si la **variable de error esta activa es porque no hay estadísticas que mostrar**, y por tanto lo primero que mostrara será un **alert con la información del error**, pero si no es así y esta iniciada la variable anteriormente mencionada que es la de las estadísticas entonces ira imprimiendo los valores que hemos ido constatando en renglones mas arriba, para finalmente borrar esas variables y dar por finalizada esta parte.

```
<div>
  <h2>Estadísticas</h2>
  <table class="table table-responsive table-secondary table-striped">
    <thead>
      <tr class="text-center">
        <th>Fecha</th>
        <th>Total Comensales</th>
        <th>Total Ingresos</th>
      </tr>
    </thead>
    <tbody>
      <?php

        // Mostramos mensaje de que no hay datos si no los hay en esas fechas

        if (isset($_SESSION['error'])) {
          echo "<tr class='text-center'>";
          echo "<td colspan='3' class='alert text-danger pb-0>No hay estadísticas de esas fechas..</td>";
          echo "</tr>";
          unset($_SESSION['error']);
        } else {

          if (isset($_SESSION['estadísticas'])) {

            // Recorremos el informe
            foreach ($_SESSION['estadísticas'] as $indice => $datos) {
              echo "<tr class='text-center'>";
              echo "<td>" . $datos['fecha'] . "</td>";
              echo "<td align='center'>" . $datos['comensales'] . "</td>";
              echo "<td align='center'>" . number_format($datos['ingresos'], 2) . " €</td>";
              echo "</tr>";
            }
            unset($_SESSION['estadísticas']);
            unset($_SESSION['fechainicial']);
            unset($_SESSION['fechafinal']);
          }
        }
      <?>
    </tbody>
  </table>
</div>
```

ahora vamos a la parte de la **lógica** de las estadísticas, que es la que gestiona **todo el tratado de los datos**, vamos a ir viéndolo en dos partes **primero de donde traen los datos**, los cuales hemos visto que mandaba **el formulario con las variables creadas en sesión y transformadas a variables globales**, y mandadas por **método post** a esta pagina

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  // Datos que vienen del post

  $fechainicial = $_POST['fechainicial'];
  $fechafinal = $_POST['fechafinal'];
  $comprobar_fecha = 0;

  $estadísticas = [];

  // Consulta pedidos pagados en rango fechas
  $consulta_pedidos = "SELECT idped, fecha
    FROM pedido
    WHERE estado=1 AND fecha BETWEEN '$fechainicial 00:00:00' AND '$fechafinal 23:59:59' ORDER BY fecha";
  $result_pedidos = mysqli_query($conn, $consulta_pedidos);
```

aquí vemos tanto la llegada de las variables como la consulta la cual obtiene los **pedidos ya terminados por día finalizado**.

Y aquí ya veríamos el resto del bucle, el cual hace un **“substring”** para solo quedarse con la **fecha del día**, ya que en la base de datos la fecha en la tabla pedidos es un **timestamp()** y eso conlleva día, horas con minutos y segundos, y como esos datos no los queremos **cortamos solo la parte del día**.

Después obtenemos el total de comensales de la tabla reserva, y por ultimo los ingresos.

Cuando ya tenemos esos datos **por día**, que lo filtra el **primer “if”**, o sea que cuando la fecha sea diferente **se le asigne a comprobar fecha de manera que con cada iteración del while la fecha ira cambiando e ira verificando que los días no se repitan**, hasta que no haya más días diferentes y acabe el while, después de todo esto, vemos como **añadimos los datos al array de estadísticas para después transformar todo en variables de sesión para hacerlo mas consistente y no perder el flujo de las variables**. Aunque no se ve y para no hacer mas capturas, esta imagen **termina en la redirección a la pagina estadísticas.php** para ver el resultado de estas búsquedas tanto si hay datos como si no, viendo como ultima instancia el **“else”** que indica un true en la variable de error lo cual como anteriormente explicado nos dice **que si esta activa** nos saldrá el error que **no hay estadísticas que mostrar**.

```
if ($result_pedidos && mysqli_num_rows($result_pedidos) > 0) {
    while ($row1 = mysqli_fetch_assoc($result_pedidos)) {
        $idped = $row1['idped'];
        $fecha = substr($row1['fecha'], 0, 10); //Obtenemos solo el dia

        if ($comprobar_fecha != $fecha) { // esta comprobacion hace que no se repitan fechas de un mismo dia
            $comprobar_fecha = $fecha; // de manera que si es diferente entonces asigna la fecha y asi obtenemos siempre una diferente

            //sacamos el numero total de comensales
            $comensales = "SELECT SUM(comensales) AS totcomen
                            FROM reserva
                            WHERE DATE(fecha) = '$fecha' AND estado=1";
            $res_comen = mysqli_query($conn, $comensales);
            $fila3 = mysqli_fetch_assoc($res_comen);
            $comensales = $fila3['totcomen'];

            // ingresos
            $ingresos = "SELECT SUM(p.precio*pp.cant) AS total_ingresos
                            FROM pedido ped, pedido_producto pp, producto p
                            WHERE ped.idped = pp.idped
                            AND pp.idprod = p.idprod
                            AND ped.estado = 1
                            AND DATE(ped.fecha) = '$fecha'";
            $res_ingre = mysqli_query($conn, $ingresos);
            $fila2 = mysqli_fetch_assoc($res_ingre);
            $ingresos = $fila2['total_ingresos'];

            // Datos en el array de estadísticas
            $estadisticas[] = [
                'fecha' => $fecha, //fecha del pedido
                'ingresos' => $ingresos, //ingreso del pedido
                'comensales' => $comensales, //comensales de la reserva
            ];
        }
    }
} else {
    $_SESSION['error'] = true;
    header('Location: estadisticas.php');
    exit();
}

// Guardar datos y fechas en sesión para mostrar en pagina estadisticas.php
$_SESSION['estadisticas'] = $estadisticas;
$_SESSION['fechainicial'] = $fechainicial;
$_SESSION['fechafinal'] = $fechafinal;
```

4.-PROBLEMAS SURGIDOS

Aunque con la consecución de este documento he ido paso a paso **desmenuzando el contenido y código** de la practica me han ido surgiendo **varios problemas** que voy detallando más adelante.

4.1- PROBLEMA CON CARRITO

Uno de los problemas surgidos era, que teniendo un pedido en el carrito, **si el cliente cerraba sesión**, al tener en la base de datos ya la unidad o **producto restado**, y esa **variable de sesión desaparecer** con la sesión, entonces las **unidades descontadas** de la base de datos quedaban en el **limbo** porque desaparecían

del carrito, estaban **descontadas del stock** y **no habían sido vendidas**, lo que hacía que en realidad tuvieras un **stock nefasto** porque no tendrías los mismos productos físicos que en la base de datos.

Una vez me di cuenta de este hecho, decidí resolverlo poniendo una **condición en el logout**, de manera que cuando le des **al botón logout**, automáticamente hará un **recorrido por la variable de sesión del carrito** y si encuentra algún producto, lo **devolverá a la base de datos** por un "update", y así nos aseguramos que cuando el carrito tenga algo y te salgas de la sesión no se queden productos en el limbo.

```
<?php
session_start();
include("conexion.php");

//hacemos una comprobacion del carrito y si tiene algo lo vaciamos para evitar que se queden cosas en el limbo

if (!empty($_SESSION['carrito'])) {
    foreach ($_SESSION['carrito'] as $producto) {
        $cantidad=$producto['cantidad'];
        $idprod=$producto['id'];
        $sql = "UPDATE producto SET stock = stock + $cantidad WHERE idprod = $idprod";
        mysqli_query($conn, $sql);
    }
}

unset($_SESSION['carrito']);
session_unset();
session_destroy();

// Redirigir al usuario a la página de inicio
header("Location:index.php");
exit();
?>
```

Y en este momento el **logout** quedaría así. Con una **condición**, que, si el carrito **no** está vacío, hará un recorrido por el, y **sumará la cantidad de cada producto a la base de datos** mediante un "update". De esta manera quedaría solventado el hecho de que cuando salgas por sesión cerrada el carrito quede vacío y los productos en el **limbo**.

4.2-PROBLEMA DE CERRAR SESION Y MESAS BLOQUEADAS

Igualmente que con el problema anterior, a lo largo de este documento esta ya resuelto pero, antes, cuando como cliente me **salía de la sesión** con un **pedido pendiente o una mesa reservada**, ocurría que **al volver**, la base de datos, salía como que **el cliente tenía una reserva** y por tanto la mesa aparecía **bloqueada**, pero no dejaba **ir a pedidos** porque la **variable de sesión de pedidos desapareció con el cierre de sesión anterior**, y por tanto le era imposible volver a saber la **id de mi pedido**, y al **no tener** id de pedido, no podía **ni ingresar** a la pestaña pedidos **ni volver a reservar mesa**, porque ya había una reserva hecha ese día **sin cerrar**, y al intentar reservar otra me enviaba a la pestaña pedidos, pero no al no saber la variable del pedido pues enviaba de nuevo a la pestaña reservar mesa y así **indefinidamente**, lo que hacía que quedase el cliente en una situación de bloqueo absoluto hasta que el camarero o desde la base de datos cerraran ese pedido, lo cual lo hacía **engorroso e inviable**.

Este problema lo resolví, haciendo una **comprobación al entrar en la pestaña mesas.php**, que si el cliente tiene un **pedido activo volviese a restaurar la sesión del numero de mesa y el id del pedido** a través de una **consulta** y después redirija al pedido.

```
// ===== Comprobamos si el usuario ya tiene un pedido activo =====
$dni_cliente = $_SESSION['dni'];
$consulta_pedido = "SELECT idped, nmesa FROM pedido WHERE usuario='$dni_cliente' AND estado=0 LIMIT 1";
$result_pedido = mysqli_query($conn, $consulta_pedido);

if ($pedido = mysqli_fetch_assoc($result_pedido)) {
    // Guardamos el idped y la mesa en sesión
    $_SESSION['idped'] = $pedido['idped'];
    $_SESSION['idmesa'] = $pedido['nmesa'];

    // Redirigimos al pedido activo
    mysqli_close($conn);
    header('Location: pedidos.php');
    exit();
}
```

Una vez en la pagina de pedidos tuve que hacer lo mismo, **verificar si hay un pedido activo** y que me muestre la lista de productos de ser así, o sea mandándome a la página de pedidos.php.

como vemos utilizo un **flag** y verifico si hay un **pedido activo o no**, si lo hay entonces **sigue el transcurso del programa**, pero si no hay un pedido activo entonces **redirige hacia mesas.php** para reservar mesa.

```
$idmesa = 'N/A';
$idped_activo = false;

if (isset($_SESSION['idped'])) {
    $idped = $_SESSION['idped'];
    $idped_activo = true;

    if (isset($_SESSION['idmesa'])) {
        $idmesa = $_SESSION['idmesa'];
    }

    $consulta = "SELECT estado FROM pedido WHERE idped = '$idped'";
    $result = mysqli_query($conn, $consulta);
    $row = mysqli_fetch_array($result);

    // Si el estado es '1' (asumiendo que significa "pedido cerrado/pagado"), redirige
    if ($row && $row['estado'] == '1') {
        unset($_SESSION['idped']);

        mysqli_close($conn);
        header('Location:mesas.php');
        exit();
    }
}

if (!$idped_activo) {
    mysqli_close($conn);
    header('Location:mesas.php'); // Redirige a donde el usuario pueda iniciar un pedido/seleccionar mesa
    exit();
}

?>
```

4.3-PROBLEMA AÑADIR AL CARRO UNA VEZ CERRADO EL PEDIDO

Uno de los problemas que también me surgieron y que solvente, es más **durante la construcción de esta memoria**, fue, que cuando el **camarero cerraba la mesa si el cliente aún no había refrescado su página entonces podría añadir algo al carrito y se restaba del stock** aunque luego lo sacara de la página porque verificaría que su mesa estaba cerrada y ya no podía pedir, pero aun así dejaba esa unidad en el carrito, la cual al volver a reservar mesa, se le **añadiría al carrito automáticamente**, algo que **no me gustaba** ni me dejaba con una sensación de profesionalidad, y lo solvente simplemente **haciendo una comprobación a la hora de añadir algo al carro**, decidí añadir también si el estado del pedido era abierto o cerrado y así no dejaría añadir nada y **redirigiría a la sección de reservar mesa automáticamente**. Por eso en las diapositivas de arriba no lo mencione porque es algo que **me di cuenta después**, y preferí incluirlo en esta parte por tanto hare una captura solo de la parte que añadí en esa página, que es la de **añadir_carro.php**

```
// --- Validamos el estado del pedido antes de añadir nada al carrito y evitar restar nada al stock ---  
if (isset($_SESSION['idped'])) {  
    $idped = $_SESSION['idped'];  
    $chequeo_pedido = mysqli_query($conn, "SELECT estado FROM pedido WHERE idped = '$idped'");  
    $row_pedido = mysqli_fetch_assoc($chequeo_pedido);  
  
    if ($row_pedido && $row_pedido['estado'] == '1') {  
  
        // Si el pedido está cerrado, limpiamos sesión y volvemos a mesas  
        unset($_SESSION['idped']);  
        mysqli_close($conn);  
        header('Location: mesas.php');  
        exit();  
    }  
}
```

Esta verificación iría situada **entre** las variables del **id del producto, la cantidad y la consulta del stock** del producto, de esta manera me aseguro que **no hace ninguna consulta mas ni añade nada** si el estado del pedido es **pagado**.

5.-BIBLIOGRAFIA

W3schools

<https://www.w3schools.com/>

Bootstrap

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>